BAN HỌC TẬP KHOA CÔNG NGHỆ PHẦN MỀM

CHUỖI TRAINING CUỐI HỌC KÌ 1 NĂM HỌC 2020 - 2021

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP

Sharing is learning

Ban học tập

Khoa Công Nghệ Phần Mềm
Trường ĐH Công Nghệ Thông Tin
ĐHQG Hồ Chí Minh

Email / Group

bht.cnpm.uit@gmail.com
www.facebook.com/groups/bht.cnpm.uit/
www.facebook.com/bhtcnpm

Training

TỔ CHỨC & CẤU TRÚC MÁY TÍNH II

Thời gian training: 10h ngày 29/01/2021

Phòng: Giảng đường 3 (A3)

Trainer: Bùi Tuấn Huy - KTPM2020

Nội dung training

A. Giới thiệu về kiến trúc máy tính

B. Kiến trúc tập lệnh

C. Vi kiến trúc

D. Chu kì thực thi lệnh

E. Hiệu suất máy tính

F. BÍ KÍP 10Đ MÔN

Giới thiệu về kiến trúc máy tính

- **Hợp ngữ và kiến trúc máy tính được chi làm 2 loại: CICS và RISC**

+ **CISC** là Complex insrtuction set computer. Đại diện tiêu biểu cho CISC là **x86** - được sử dụng trên các máy tính cá nhân và server

+ **RICS** là Reduced instruction set computer. Đại diện cho RISC là **ARM** và **MIPS**. ARM được sử dụng trong các thiết bị di động và MIPS được sử dụng trong một số siêu máy tính, và các thiết bị như router

Giới thiệu về kiến trúc máy tính

- Khác biệt giữa CISC và RISC:

+ Hợp ngữ của **CISC** rất phức tạp và các máy CISC tiêu tốn điện năng

+ Hợp ngữ của **RISC** thì đơn giản hơn, vì vậy các máy **RISC** tiêu tốn điện năng ít hơn

Hãy nêu tên 1 vài thiết bị sử dụng ứng với từng loại kiến trúc??

- x86: Các máy tính, laptop sử dụng chip của Intel
- ARM: Các smartphone của các hãng như Apple, Samsung,..
- MIPS: Nintendo 64, Sony Playstation 2

Kiến trúc tập lệnh

I. Các định nghĩa chung

- **Lệnh** (Instruction) là một chỉ dẫn để máy tính tính thực hiện công việc nào đó
- **Tập lệnh** (Instruction Set) là tập hợp các lệnh của máy tính
- **Kiến trúc Tập lệnh = Tập lệnh + Biểu diễn lệnh**
- **Kiến trúc tập lệnh:**
    + Tập lệnh: Máy tính có thể làm những gì?
    + Định dạng lệnh (biểu diễn lệnh): Mỗi lệnh được biểu diễn như thế nào?
        - Opcode (Operation Code): Mã thao tác
        - Toán hạng: Các toán hạng cần thiết để thực hiện thao tác
        - Các trường khác…
- **Phân loại:**
    + Ngăn xếp (stack)
    + Bộ tích lũy (accumulator)
    + Thanh ghi – Bộ nhớ (register–memory)
    + Thanh ghi – thanh ghi / nạp – lưu (register-register/load-store)

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

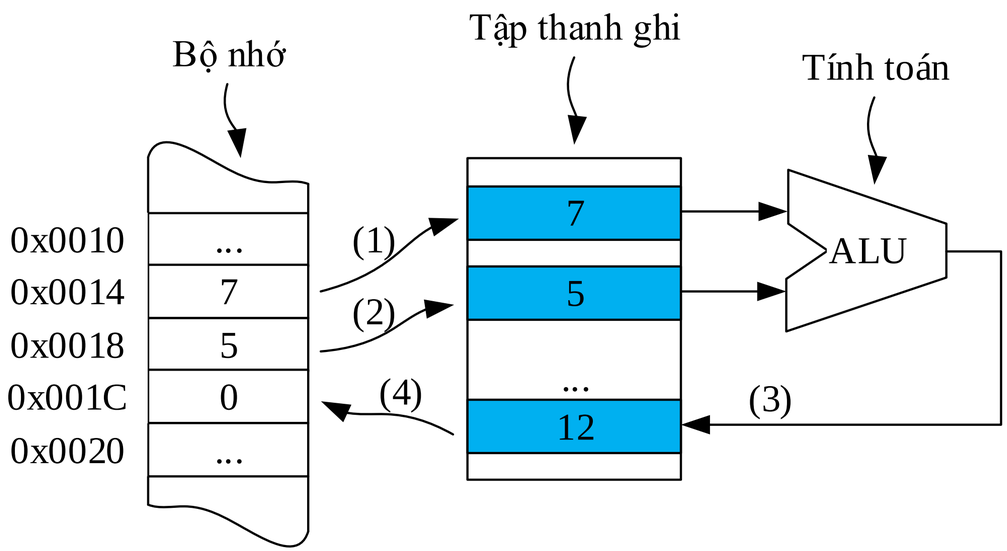
Kiến trúc tập lệnh

II. Các định nghĩa của MIPS

- Thiết kế theo kiến trúc thanh ghi – thanh ghi
- Độ rộng lệnh: Cố định 32 bit cho tất cả các lệnh
- Định dạng lệnh: R, I, J
- Tập thanh ghi: 32 thanh ghi 32 bit, thanh ghi $zero luôn bằng 0
- Kiểu dữ liệu: Byte (8 bit), halfword (16 bit), word (32 bit)
- Chế độ định địa chỉ: 5 chế độ
- Toán hạng: Thanh ghi, số tức thời (bù 2), bộ nhớ
- Định địa chỉ theo byte

Kiến trúc tập lệnh

III. Kiến trúc thanh ghi – thanh ghi



- Dữ liệu được lưu trữ ở bộ nhớ
- Tính toán trên thanh ghi (không tính toán trên bộ nhớ)
- Cần nạp dữ liệu từ bộ nhớ vào thanh ghi để tính toán
- Cần lưu giá trị thanh ghi vào bộ nhớ sau khi tính toán

Kiến trúc tập lệnh

IV. Toán hạng

- **Toán hạng** là một dữ liệu được dùng để tính toán
- MIPS có 3 loại toán hạng:
  + Toán hạng **thanh ghi**: Dữ liệu nằm trong thanh ghi
  + Toán hạng **bộ nhớ**: Dữ liệu nằm trong bộ nhớ
  + Toán hạng **số tức thời**: Dữ liệu nằm ngay trong lệnh

Kiến trúc tập lệnh

V. Toán hạng thanh ghi

- Tên gợi nhớ: Tiền tố $ theo sau là chỉ số hoặc tên ($2 hay $sp)
  + $t0, $t1, ..., $t9 cho các dữ liệu tạm
  + $s0, $s1, ..., $s7 cho lưu trữ các biến
  + $v0, $v1, $k1, ... cho các mục đích đặc biệt khác

- Thanh ghi trong hợp ngữ không có kiểu dữ liệu, cách ta sử dụng thanh ghi sẽ quyết định kiểu dữ liệu là gì

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Kiến trúc tập lệnh

V. Toán hạng thanh ghi

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
$zero	0	The Constant Value 0	N.A.
$at	1	Assembler Temporary	No
$v0-$v1	2-3	Values for Function Results and Expression Evaluation	No
$a0-$a3	4-7	Arguments	No
$t0-$t7	8-15	Temporaries	No
$s0-$s7	16-23	Saved Temporaries	Yes
$t8-$t9	24-25	Temporaries	No
$k0-$k1	26-27	Reserved for OS Kernel	No
$gp	28	Global Pointer	Yes
$sp	29	Stack Pointer	Yes
$fp	30	Frame Pointer	Yes
$ra	31	Return Address	Yes

Kiến trúc tập lệnh

VI. Toán hạng bộ nhớ

- Bộ nhớ được đánh địa chỉ theo byte
- MIPS quy định địa chỉ bộ nhớ phải là bội số của 4
- MIPS sử dụng mô hình địa chỉ Big-Endian
- **Big-Endian** sắp xếp các byte vào bộ nhớ theo **ngược lại** ví trị của byte khi biểu diễn trong 1 word
- **Big-Endian:** Tức là byte cao sẽ được lưu ở địa chỉ thấp
- VD: Lưu trữ số nguyên 7411 vào bộ nhớ máy trên nền tảng dùng big endian. Số 7411 được viết dưới dạng nhị phân như sau:

1 1100 1111 0011

Minh họa việc lưu số 7411 vào bộ nhớ máy tính như sau:

...	00000000	00000000	00011100	11110011	...
	1000	1001	1002	1003	

Kiến trúc tập lệnh

VII. Toán hạng tức thời

- Dữ liệu hằng số được chỉ định ngay trong lệnh
  + Sử dụng dữ liệu ngay mà không cần tìm kiếm như thanh ghi và bộ nhớ
    - ➢ Không cần phải nạp dữ liệu từ bộ nhớ!!!
    - ➢ Giá trị thường nhỏ
- Ví dụ:

  addi $s3, $s2, 4

  addi $t2, $t1, -7
- MIPS có thanh ghi số 0 ($zero) luôn luôn là một hằng số 0

  Sao chép giá trị: **add $t2, $t1, $zero**

Kiến trúc tập lệnh

VIII. Định dạng lệnh

- Lệnh được biểu diễn bằng các mã nhị phân (mã máy)
- Định dạng lệnh là một hình thức biểu diễn một lệnh dưới dạng các trường mã nhị phân
    +Lệnh của MIPS đều rộng 32 bit
- MIPS có 3 định dạng lệnh:
    + Định dạng **lệnh R**: Cho các thao tác tuần túy trên thanh ghi
    + Định dạng **lệnh I**: Cho các thao tác sử dụng số tức thời có giá trị nhỏ và vừa
    + Định dạng **lệnh J**: Cho các thao tác sử dụng số tức thời có giá trị rất lớn

Kiến trúc tập lệnh

IX. Định dạng lệnh R

op	rs	rt	rd	shamt	funct
6 bit	5 bit	5 bit	5 bit	5 bit	6 bit

- Các trường lệnh
  + **op (opcode)**: Mã thao tác
  + **rs**: Địa chỉ toán hạng thanh ghi nguồn thứ nhất
  + **rt**: Địa chỉ toán hạng thanh ghi nguồn thứ hai
  + **rd**: Địa chỉ toán hạng thanh ghi đích
  + **shamt** (shift amount): Lượng dịch (hiện tại là00000)
  + **funct** (function code): Mã thao tác (op) mở rộng

IX. Định dạng lệnh R

Biểu diễn lệnh **add $t0, $t5, $s7**

op (add)	rs ($t5)	rt ($s7)	rd ($t0)	shamt (0)	funct (add)
0	13	23	8	0	0x20
000000	01101	10111	01000	00000	100000

0000 0001 1011 0111 0100 0000 0010 0000 **0x01B74020**

Kiến trúc tập lệnh

X. Định dạng lệnh I

op	rs	rt	immediate
6 bit	5 bit	5 bit	16 bit

- Các trường lệnh
  + op (opcode): Mã thao tác
  + rs: Địa chỉ toán hạng thanh ghi nguồn thứ nhất
  + rt: Địa chỉ toán hạng thanh ghi nguồn thứ hai hoặc thanh ghi đích
  + immediate: Số tức thời 16 bit (biểu diễn dạng bù 2)
    ✓Thường là độ dời

Kiến trúc tập lệnh

X. Định dạng lệnh I

Biểu diễn lệnh **lw $a0, 48($s2)**

op (lw)	rs ($s2)	rt ($a0)	Immediate (48)
0x23	18	4	48
100011	10010	00100	0000000000110000

1000 1110 0100 0100 0100 0000 0011 0000 **0x8E444030**

Kiến trúc tập lệnh

XI. Định dạng lệnh J

op	address
6 bit	26 bit

- Các trường lệnh
  + op (opcode): Mã thao tác
  + address: Số tức thời 26 bit (biểu diễn dạng bù 2)
    ✓Bit [27:2] của địa chỉ nhảy tới

Vi kiến trúc

- Kiến trúc Máy tính bao gồm 3 thành phần chính:
+ Kiến trúc tập lệnh (ISA)
+ Vi kiến trúc (Tổ chức Phần cứng Máy tính)
+ Hệ thống Máy tính
- Về chức năng, Vi kiến trúc là một tổ chức phần cứng dùng để hiện thực tập lệnh của một máy tính.
- Về cấu tạo, Vi kiến trúc được chia thành 2 khối:
    + Khối đường dữ liệu (datapath): Thực thi lệnh
    + Khối điều khiển (control unit): Điều khiển datapath hoạt động

Chu kì thực thi lệnh

- Bộ nhớ lệnh
- PC

- Tập thanh ghi
- Bộ nhớ dữ liệu
- Mở rộng dấu

- ALU
- Bộ so sánh

- Bộ nhớ dữ liệu

- Tập thanh ghi

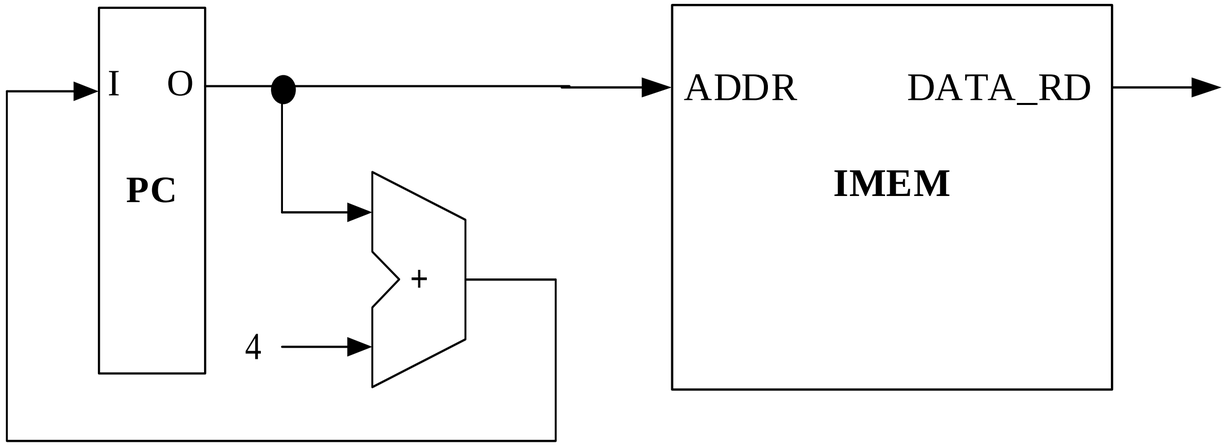
Chu kì thực thi lệnh

I. Nạp lệnh

- Lệnh cần nạp lưu trong Bộ nhớ lệnh

- Địa chỉ của lệnh cần nạp lưu trong thanh ghi PC

+ Tăng PC lên 4 để chuẩn bị nạp lệnh tiếp theo -> cần thêm bộ cộng

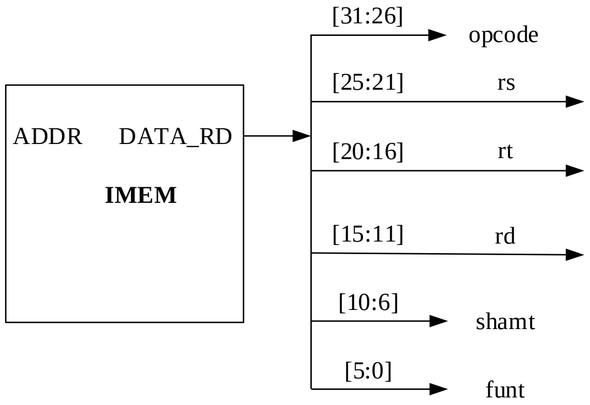


Chu kì thực thi lệnh

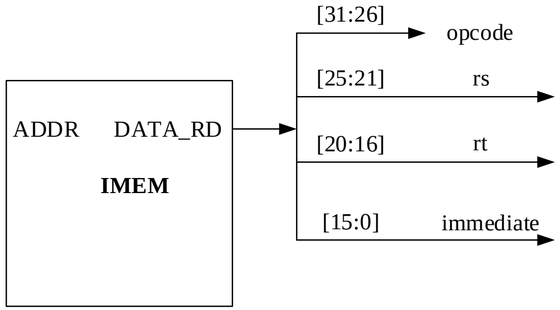
II. Giải mã lệnh – Định dạng lệnh

- Dựa vào opcode để xác định định dạng lệnh

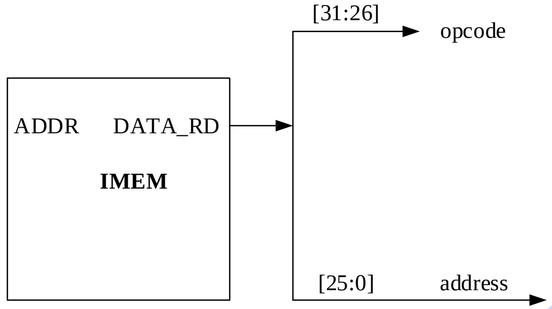
Giải mã các lệnh có **định dạng R**



Giải mã các lệnh có **định dạng I**



Giải mã các lệnh có **định dạng J**

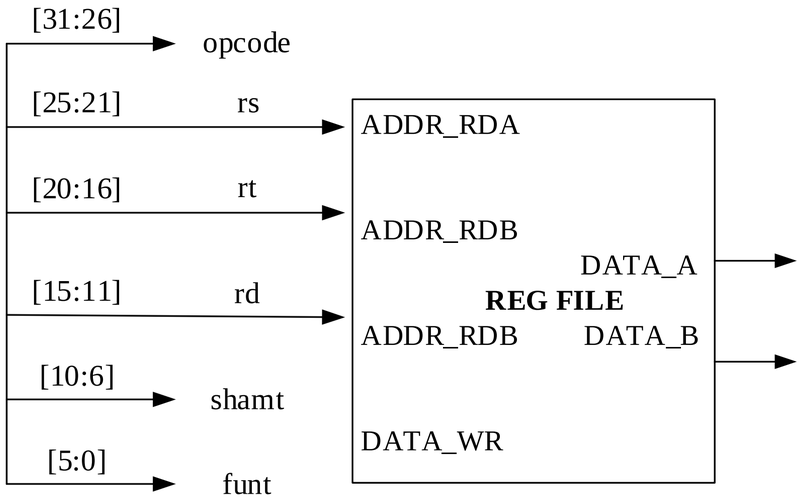


Chu kì thực thi lệnh

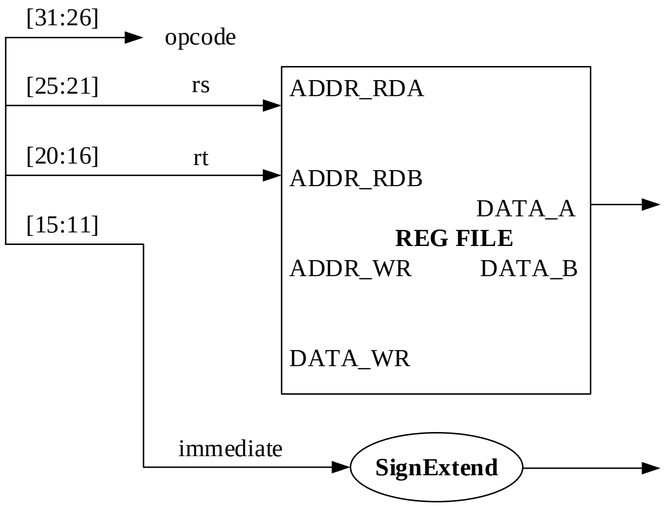
III. Giải mã lệnh - Nạp toán hạng

- Dựa vào định dạng lệnh mà nạp toán hạng tương ứng, các lệnh khác nhau sẽ nạp các toán hạng khác nhau, slide chỉ minh họa 2 trường hợp nạp toán hạng

Nạp toán hạng cho các lệnh có **2 toán hạng nguồn đều là thanh ghi** (chẳng hạn như lệnh add, and, ...)

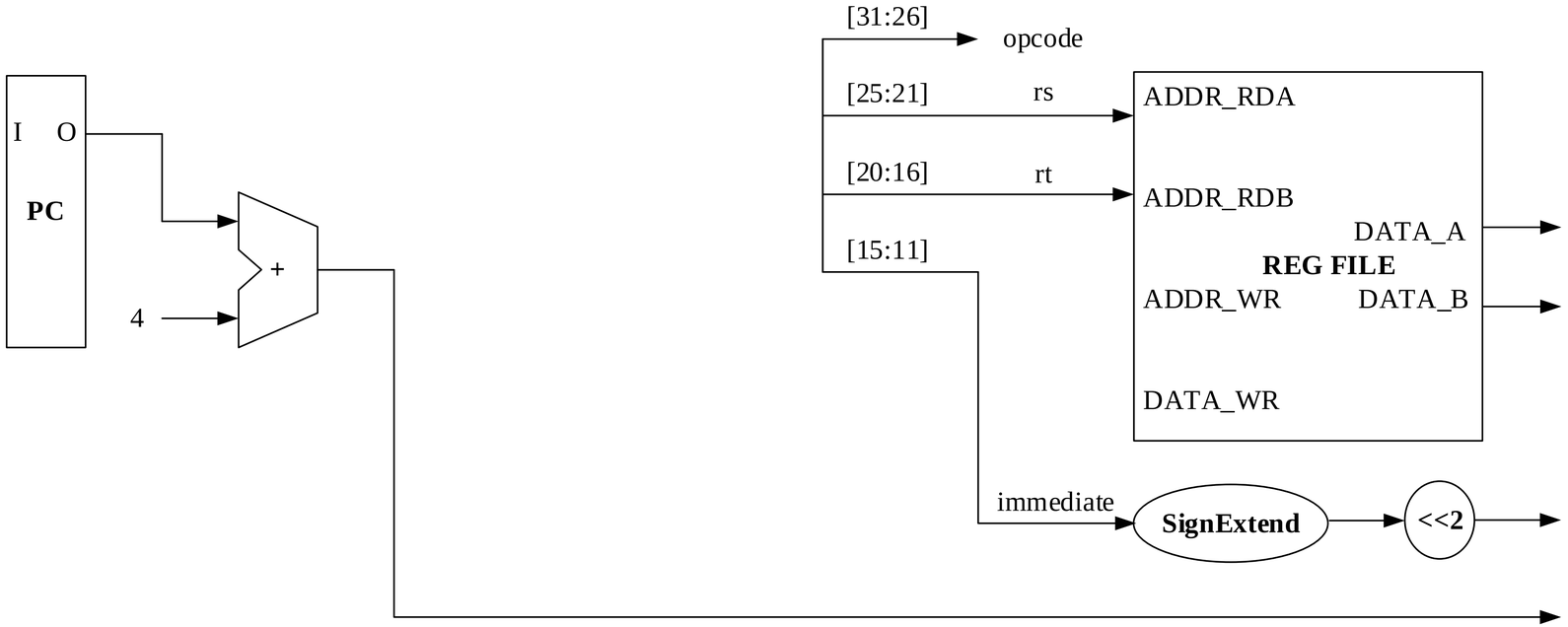


Nạp toán hạng cho các lệnh có **1 toán hạng nguồn là thanh ghi và 1 toán hạng nguồn là số tức thời** (chẳng hạn như lệnh addi, lw, ...)



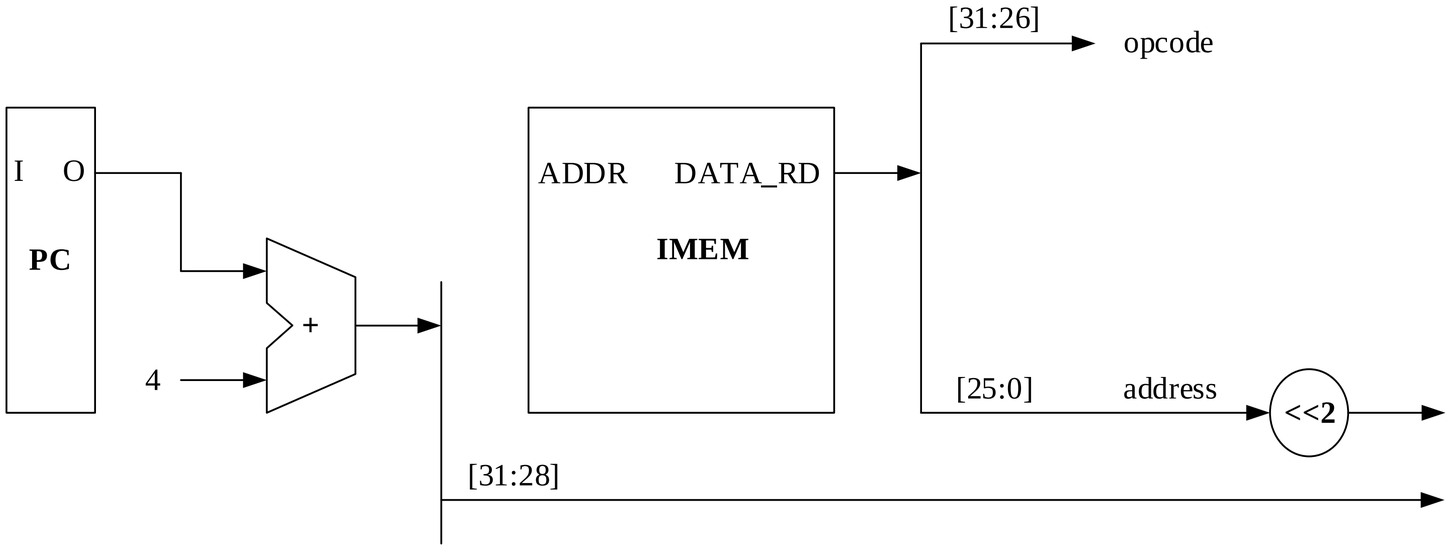
Chu kì thực thi lệnh

III. Giải mã lệnh - Nạp toán hạng: beq/bne



Chu kì thực thi lệnh

III. Giải mã lệnh - Nạp toán hạng: j

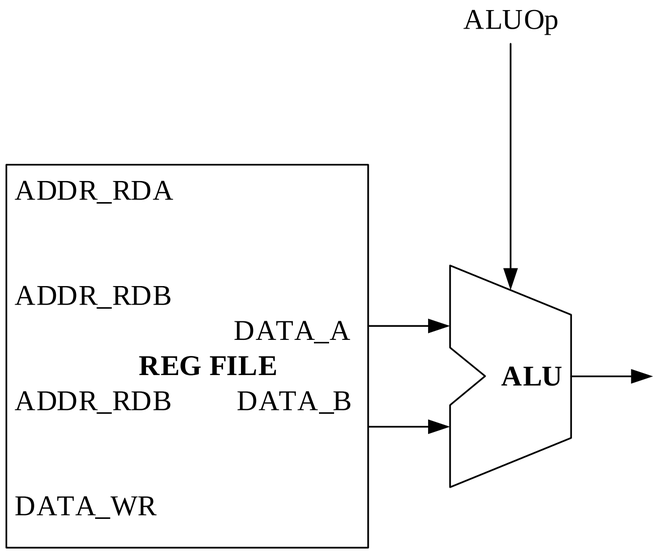


Chu kì thực thi lệnh

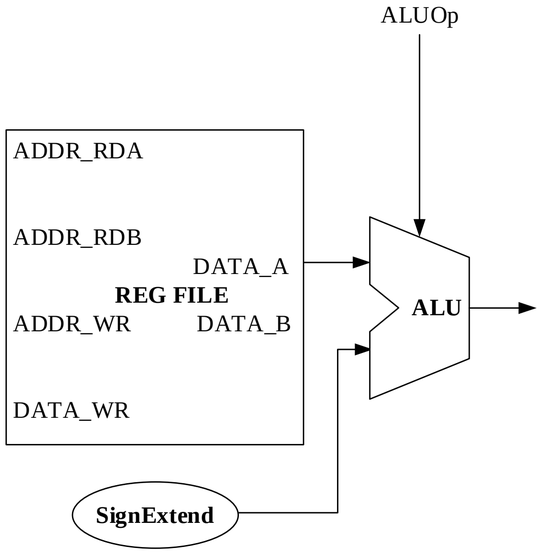
IV. Thực thi

Dựa vào opcode và funt mà quyết định thao tác gì sẽ được thực thi

Thực thi đối với các lệnh có **2 toán hạng nguồn đều nằm trên thanh ghi** (chẳng hạn như add, beq, ...)



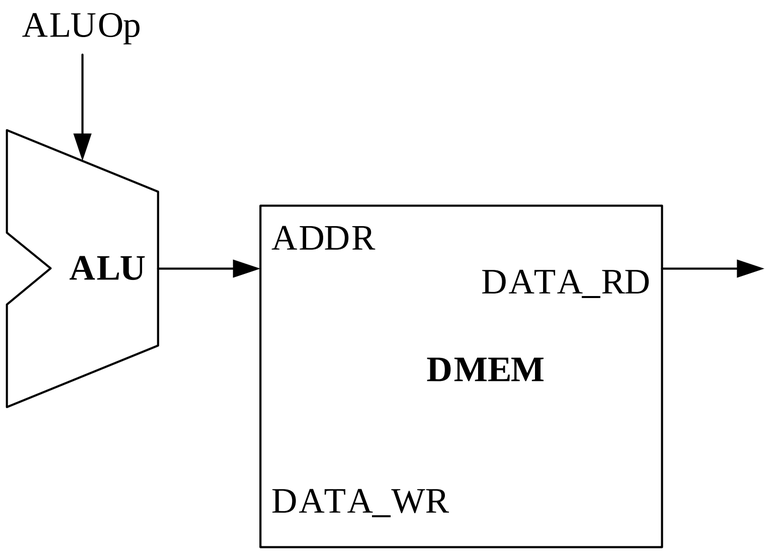
Thực thi đối với các lệnh có **1 toán hạng nguồn là thanh ghi, 1 toán hạng nguồn là số tức thời** (chẳng hạn như addi, lw, ...)

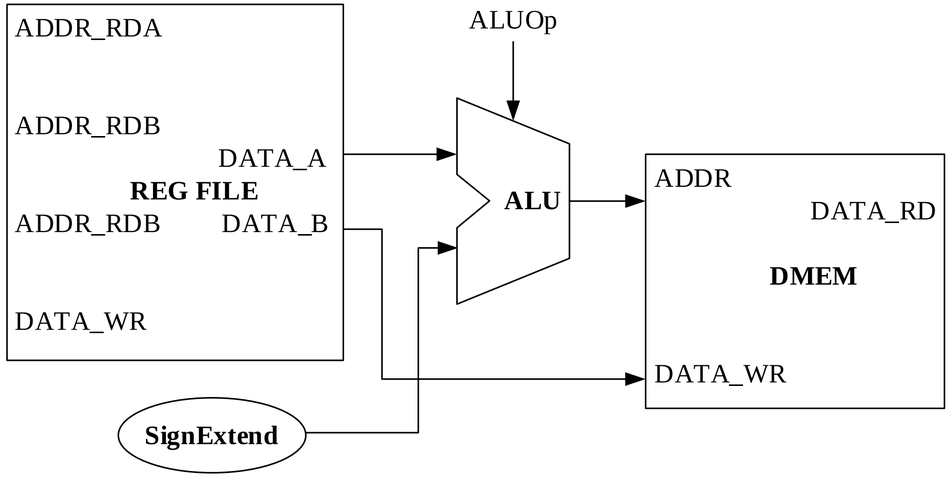


Chu kì thực thi lệnh

V. Truy xuất bộ nhớ

- Đọc dữ liệu từ bộ nhớ dữ liệu đối với các lệnh nạp
- Ghi dữ liệu tới bộ nhớ dữ liệu đối với các lệnh lưu



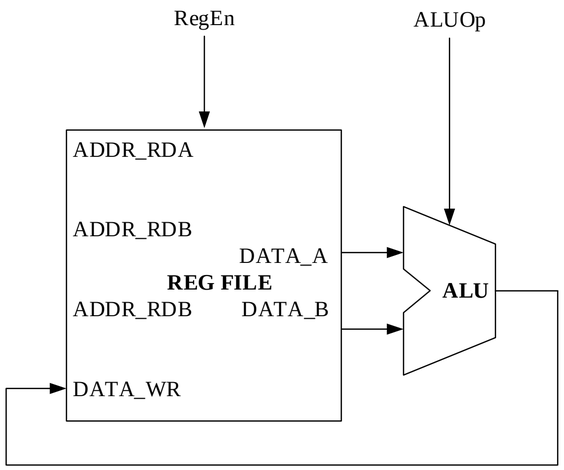


Chu kì thực thi lệnh

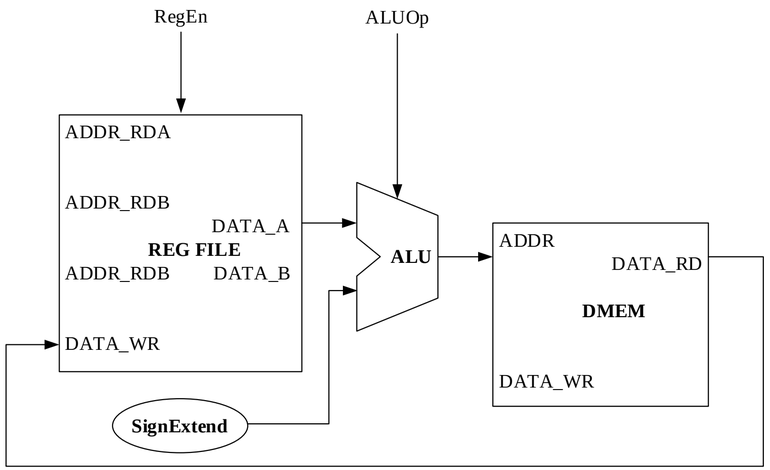
VI. Lưu kết quả

- Có thể ghi dữ liệu về lại Tập thanh ghi
- Tín hiệu RegEn quyết định việc lưu kết quả có diễn ra hay không

Lưu kết quả cho các lệnh tính toán thuần túy trên thanh ghi (chẳng hạn như add, or, ...)



Lưu kết quả cho các lệnh nạp dữ liệu (chẳng hạn như lw, lbu, ...)



Hiệu suất máy tính

I. Thời gian thực thi và hiệu suất

- Thời gian thực thi: Tổng thời gian để hoàn thành một tác vụ nào đó
  + Truy cập ổ đĩa, bộ nhớ, …
- Hiệu suất: Số lượng tác vụ hoàn thành trong một đơn vị thời gian

$$\text{Hiệu suất} = \frac{1}{\text{Thời gian thực thi}}$$

$$\text{Hiệu suất}_X > \text{Hiệu suất}_Y$$

$$\frac{1}{\text{Thời gian thực thi}_X} > \frac{1}{\text{Thời gian thực thi}_Y}$$

$$\text{Thời gian thực thi}_X < \text{Thời gian thực thi}_Y$$

Hiệu suất máy tính

II. Clock

- Máy tính cần một clock để xác định khi nào một thao tác được thực hiện trong phần cứng.

- Hai khái niệm liên quan đến clock:

  + **Chu kỳ** (Clock cycle)

  + **Tần số** (Clock rate hoặc clock frequency)

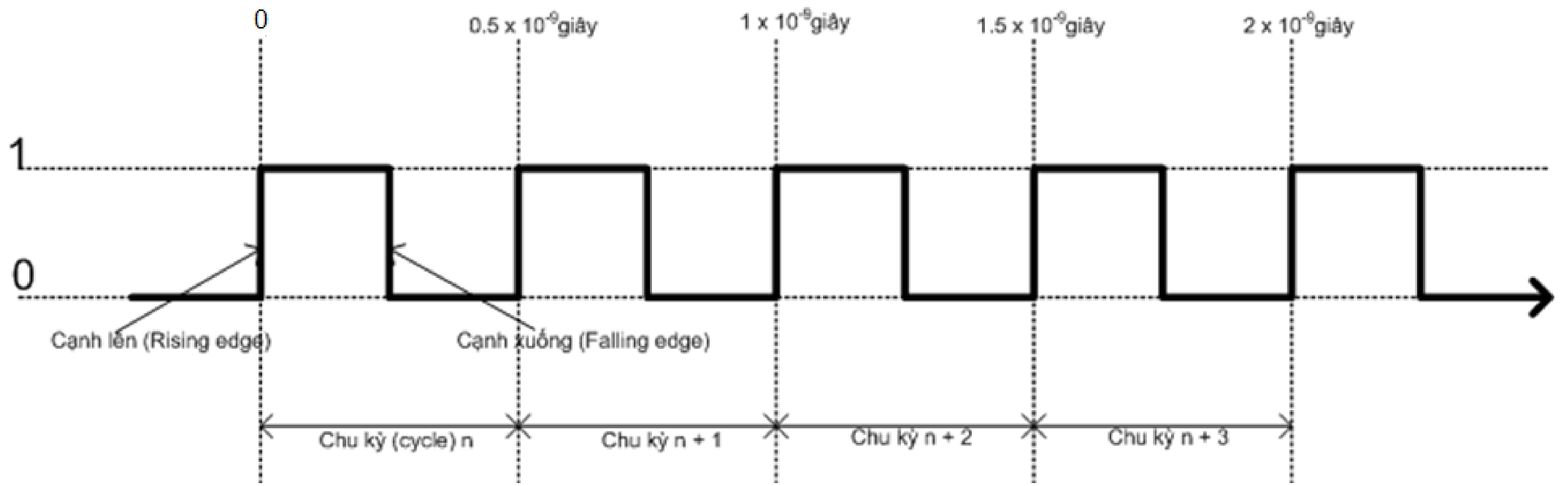
Thời gian thực thi = Tổng số chu kỳ clock * Chu kỳ clock

$$\textbf{Thời gian thực thi} = \frac{\textbf{Tổng số chu kỳ clock}}{\textbf{Tần số clock}}$$

Tăng hiệu suất bằng cách giảm chu kỳ clock (tăng tần số clock)

Hiệu suất máy tính

II. Clock



Chu kỳ clock = 0.5×10^{-9} giây
(Clock cycle time/clock cycle/ cycle time)

Tần số clock (Clock rate) = $\frac{1}{\text{Chu kỳ clock}} = \frac{1}{0.5 \times 10^{-9}} = 2 \times 10^{9}$ Hz = 2 GHz

Hiệu suất máy tính

II. Clock

Máy tính A chạy ở tần số 3 Ghz cần 12s để hoàn thành chương trình P. Máy tính B chỉ cần 10s để hoàn thành chương trình P nhưng tổng số chu kỳ cần để hoàn thành chương trình P nhiều gấp 1.2 lần so với máy tính A.

Máy tính B chạy ở tần số bao nhiêu?

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Hiệu suất máy tính

III. CPI (Clock cycle Per Instruction)

Tổng số chu kỳ clock = Tổng số lệnh * CPI

Thời gian thực thi = Tổng số lệnh * CPI * Chu kỳ clock

$$\text{Thời gian thực thi} = \frac{\text{Tổng số lệnh} * \text{CPI}}{\text{Tần số clock}}$$

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Hiệu suất máy tính

III. CPI (Clock cycle Per Instruction)

Máy tính A: Chu kỳ clock = 250ps, CPI = 2.5
Máy tính B: Chu kỳ clock = 500ps, CPI = 1.25
Cả 2 máy tính đều có cùng ISA. Máy tính nào nhanh hơn và nhanh hơn bao nhiêu lần?

Hiệu suất máy tính

III. CPI (Clock cycle Per Instruction)

Mỗi tập lệnh có nhiều nhóm lệnh khác nhau

$$\text{Chu kỳ clock} = \sum_{i=1}^{n} \left(\text{CPI}_i * \text{Số lệnh trong nhóm}_i\right)$$

$$\text{CPI} = \frac{\text{chu kỳ clock}}{\text{Tổng số lệnh}} = \sum_{i=1}^{n} \left(\text{CPI}_i * \frac{\text{Số lệnh trong nhóm}_i}{\text{Tổng số lệnh}}\right)$$

Hiệu suất máy tính

III. CPI (Clock cycle Per Instruction)

Có 2 cách biên dịch chương trình bằng cách sử dụng các nhóm lệnh A, B, C như bảng dưới.

- Cách biên dịch nào tạo ra tổng số lệnh nhỏ hơn?
- Cách biên dịch nào tạo ra chương trình chạy nhanh hơn? CPI là bao nhiêu?

Nhóm lệnh	**A**	**B**	**C**
CPI cho mỗi nhóm	1	3	2
Số lệnh cho cách 1	3	2	2
Số lệnh cho cách 2	4	2	1

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Hiệu suất máy tính

IV. Các yếu tố ảnh hưởng đến hiệu suất

- Các **thành phần** của hiệu suất:
  - + **Thời gian thực thi**
  - + **Tổng số lệnh**
  - + **Tần số**
  - + **CPI**

Yếu tố phần cứng/phần mềm	Tác động vào gì?
Thuật toán	Tổng số lệnh, và có thể cả CPI
Ngôn ngữ lập trình	Tổng số lệnh, CPI
Trình biên dịch	Tổng số lệnh, CPI
Kiến trúc tập lệnh	Tổng số lệnh, tần số, CPI

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Hiệu suất máy tính

V. Các kỹ thuật nâng cao hiệu suất

- Giảm thời gian thực thi
  + **Tăng tần số clock** (Bị giới hạn bởi phần cứng)
  + **Pipeline**: Thực thi đồng thời **nhiều lệnh** bằng cách chia chu kỳ thực thi lệnh thành các stage. Tại một thời điểm, một lệnh chỉ được thực thi một stage
  + **Tiên đoán**: Dự đoán việc nhảy (các lệnh nhảy) có xảy ra hay không
  + **Multicore**: Thực thi đồng thời **nhiều chương trình** bằng cách tăng số lượng bộ xử lý
  + **Multithread**: Thực thi đồng thời **nhiều tác vụ** bằng cách tăng số lượng đơn vị xử lý
  + **Phân cấp bộ nhớ**: Sử dụng các bộ nhớ nhanh cho việc thao tác với dữ liệu, sử dụng các bộ nhớ chậm cho việc lưu trữ dữ liệu, …

BÍ KÍP 10Đ MÔN

QUAN TRỌNG

Bảng tra cứu MIPS

Đề thi và cách giải

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

BAN HỌC TẬP KHOA CÔNG NGHỆ PHẦN MỀM
CHUỖI TRAINING CUỐI HỌC KÌ 1 NĂM HỌC 2020 - 2021

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP

Sharing is learning

HẾT

CẢM ƠN CÁC BẠN ĐÃ THEO DÕI.
CHÚC CÁC BẠN CÓ KẾT QUẢ THI THẬT TỐT!

Ban học tập

Khoa Công Nghệ Phần Mềm
Trường ĐH Công Nghệ Thông Tin
ĐHQG Hồ Chí Minh

Email / Group

bht.cnpm.uit@gmail.com
www.facebook.com/groups/bht.cnpm.uit/
www.facebook.com/bhtcnpm